



รายงาน Mini Project
เรื่อง ระบบแชทออนไลน์ (Online Chat System)

จัดทำโดย
นางสาวเพียงตา จันทรประโคน ปี2 หมู่1 รหัสนักศึกษา 670112418030

เสนอ
ผู้ช่วยศาสตราจารย์ ดร.สวิน วงศ์ประเมษฐ์

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา เทคโนโลยีอินเทอร์เน็ต (4132203)
ภาคเรียนที่ 2 ปีการศึกษา 2568
สาขาวิชาเทคโนโลยีสารสนเทศ คณะวิทยาศาสตร์ มหาวิทยาลัยราชภัฏบุรีรัมย์

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา เทคโนโลยีอินเทอร์เน็ต (4132203) จัดทำขึ้นเพื่อศึกษาและพัฒนา ระบบแชทออนไลน์ (Online Chat System) ซึ่งเป็นเทคโนโลยีที่มีความสำคัญในปัจจุบัน โดยมีวัตถุประสงค์เพื่อให้ผู้ใช้สามารถสื่อสารกันได้อย่างรวดเร็ว มีประสิทธิภาพ และปลอดภัย

เนื้อหาในรายงานนี้ประกอบด้วย ที่มาและความสำคัญของระบบ หลักการทำงาน เครื่องมือที่ใช้ และผลการดำเนินงาน ซึ่งนำเสนอเทคโนโลยีสำคัญที่ใช้ในการพัฒนา ระบบแชทออนไลน์ (Online Chat System) ได้แก่ Node.js, MySQL, Socket.IO, Express และ Docker เพื่อรองรับการทำงานของระบบแชทแบบเรียลไทม์ (Real-time Communication) ให้สามารถส่งและรับข้อความได้ทันทีระหว่างผู้ใช้งานผ่านเครือข่ายอินเทอร์เน็ต

ผู้จัดทำหวังว่ารายงานฉบับนี้จะเป็นประโยชน์แก่ผู้ที่สนใจศึกษาเกี่ยวกับเทคโนโลยีอินเทอร์เน็ตและสามารถนำความรู้ที่ได้รับไปประยุกต์ใช้ในการพัฒนาระบบแชทหรือแอปพลิเคชันอื่น ๆ ต่อไป

ผู้จัดทำ

นางสาวเพียงตา จันทร์ประโคน

เนื้อหา

ระบบแชทออนไลน์คืออะไร

ระบบแชทออนไลน์ (Online Chat System) คือระบบที่ช่วยให้ผู้ใช้สามารถสื่อสารกันแบบเรียลไทม์ผ่านเครือข่ายอินเทอร์เน็ต โดยสามารถส่งข้อความ ตัวอ็อบเจกต์ รูปภาพ หรือไฟล์ต่าง ๆ ได้อย่างรวดเร็ว ตัวอย่างเช่น LINE, Facebook Messenger, Discord และ Telegram

การทำงานของระบบแชทจะอยู่ในรูปแบบ Client-Server ซึ่งฝั่งผู้ใช้ (Client) จะส่งข้อความไปยังเซิร์ฟเวอร์ และเซิร์ฟเวอร์จะกระจายข้อความนั้นไปยังผู้รับปลายทาง

โครงสร้างของระบบแชท

โครงสร้างของระบบแชทออนไลน์สามารถแบ่งออกเป็น 3 ส่วนหลัก ได้แก่

1. ผู้ส่งข้อความ (Sender / Client A)

ผู้ใช้ที่เริ่มต้นการสนทนา โดยพิมพ์ข้อความผ่านหน้าเว็บหรือแอปพลิเคชัน แล้วส่งข้อมูลไปยังเซิร์ฟเวอร์

2. เซิร์ฟเวอร์กลาง (Chat Server)

ทำหน้าที่รับข้อความจากผู้ส่ง และกระจายข้อความนั้นไปยังผู้รับแบบเรียลไทม์ โดยเซิร์ฟเวอร์จะใช้เทคโนโลยี เช่น WebSocket หรือ Socket.IO เพื่อให้ผู้ใช้ทุกคนที่เชื่อมต่ออยู่สามารถรับข้อความพร้อมกันได้ทันที

3. ผู้รับข้อความ (Receiver / Client B)

ผู้ใช้ที่กำลังเชื่อมต่อกับระบบ และจะได้รับข้อความจากผู้ส่งแบบทันทีโดยไม่ต้องรีเฟรชหน้าเว็บ

ระบบแชทออนไลน์ใช้หลักการทำงานแบบ Client-Server ซึ่งช่วยให้การสื่อสารเกิดขึ้นแบบสองทาง (Full Duplex) คือผู้ส่งและผู้รับสามารถส่ง-รับข้อมูลได้พร้อมกันในเวลาเดียวกัน

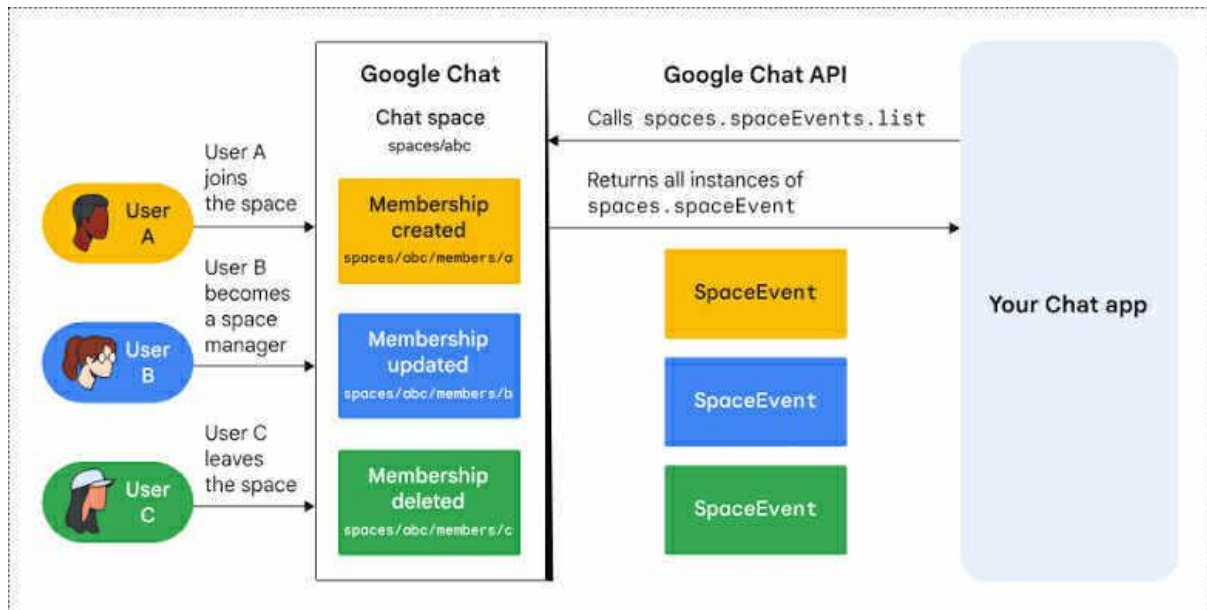
ลำดับขั้นตอนการทำงานของระบบแชท

1. ผู้ใช้ A เปิดหน้าแชทและเชื่อมต่อกับ Chat Server ผ่าน WebSocket
2. เมื่อผู้ใช้ A พิมพ์ข้อความและกดส่ง → ข้อความถูกส่งไปยัง Server
3. Chat Server จะตรวจสอบห้องแชท (Chat Room) และส่งข้อความต่อไปยังผู้ใช้ B ที่อยู่ในห้องเดียวกัน

4. ผู้ใช้ B จะเห็นข้อความนั้นทันทีในหน้าจอของตนเอง

แผนภาพโครงสร้างระบบเซต

ជួប A (Client) $\rightleftharpoons$ Chat Server $\rightleftharpoons$ ជួប B (Client)



ประวัติความเป็นมาของระบบแซท

- ปี 1988 — ระบบแชทออนไลน์เริ่มต้นในบริการ IRC (Internet Relay Chat)
- ปี 1996 — มีการพัฒนาโปรแกรมสนทนาแบบ GUI เช่น ICQ
- ปี 2000 — โปรแกรม MSN Messenger และ Yahoo Messenger ได้รับความนิยม
- ปี 2009 — เริ่มมีการใช้เทคโนโลยี WebSocket สำหรับการแชทบนเว็บไซต์
- ปัจจุบัน — ระบบแชทพัฒนาให้สามารถส่งภาพ เสียง วิดีโอ และรองรับการเข้ารหัสข้อมูล (Encryption) เพื่อความปลอดภัย

องค์ประกอบหลักของระบบแซท

1. Client (ผู้ใช้) – ส่วนติดต่อผู้ใช้ เช่น หน้าเว็บหรือแอปพลิเคชันมือถือ
2. Chat Server – ส่วนประมวลผลและกระจายข้อความ

3. Database – สำหรับเก็บข้อความและข้อมูลผู้ใช้
4. WebSocket / Socket.IO – โปรโตคอลที่ใช้ในการสื่อสารแบบเรียลไทม์
5. API (REST/HTTP) – ใช้ในการดึงข้อมูลผู้ใช้หรือประวัติการแชท

โปรโตคอลที่เกี่ยวข้อง

- HTTP (Hypertext Transfer Protocol)

ใช้สำหรับส่งข้อมูลทั่วไป เช่น สมัครสมาชิก ล็อกอิน หรือโหลดประวัติแชท

- WebSocket

เป็นโปรโตคอลที่พัฒนาเพิ่มจาก HTTP เพื่อให้สามารถเชื่อมต่อแบบเรียลไทม์ได้ทั้งสองทาง (Full Duplex)

พอร์ตที่ใช้งาน:

- 80 (HTTP)
- 443 (HTTPS / WSS)

ขั้นตอนการทำงานของ WebSocket:

1. ผู้ใช้เปิดการเชื่อมต่อกับ Chat Server
2. Server สร้าง “Session” สำหรับผู้ใช้
3. เมื่อมีการส่งข้อความ ข้อความจะถูกส่งไปยัง Server
4. Server ส่งข้อความนั้นไปยังผู้รับทันที

การใช้งานร่วมกับภาษาโปรแกรม

ตัวอย่าง โค้ด Node.js + Socket.IO

```
// server.js
```

```
const express = require("express");
```

```
const http = require("http");
```

```
const { Server } = require("socket.io");
```

```

const app = express();

const server = http.createServer(app);

const io = new Server(server);

io.on("connection", (socket) => {

  console.log("มีผู้ใช้เชื่อมต่อ");

  socket.on("chat message", (msg) => {

    io.emit("chat message", msg);

  });

});

server.listen(3000, () => {

  console.log("Server running on port 3000");

});

```

คำอธิบาย

- ใช้ Express สร้าง HTTP Server
- ใช้ Socket.IO สำหรับรับ-ส่งข้อความแบบเรียลไทม์

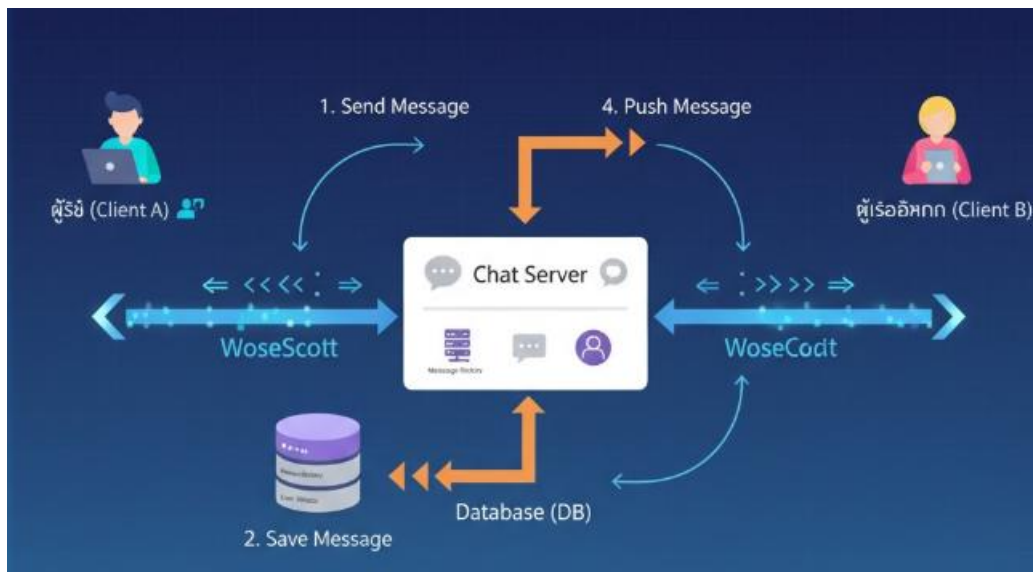
พอร์ตที่ใช้งาน

โปรโตคอล	พอร์ต	การใช้งาน
HTTP	80	ใช้โหลดหน้าเว็บ
HTTPS	443	ใช้โหลดหน้าเว็บแบบเข้ารหัส
WebSocket (WS/WSS)	3000 / 8080 / 443	สำหรับแชทแบบเรียลไทม์

ภาพประกอบ

ภาพโครงสร้างระบบแชท

ผู้ใช้ (Client) ↔ WebSocket ↔ Chat Server ↔ Database ↔ ผู้ใช้อีกคน



การใช้งาน Chat Server บน Docker

ตัวอย่าง Image

Image	รายละเอียด
node:18-alpine	ใช้รัน Node.js
redis	ใช้เก็บข้อความชั่วคราว

ตัวอย่าง docker-compose.yml

version: '3.8'

services:

chatapp:

image: node:18-alpine

container_name: chat-server

ports:

- "3000:3000"

volumes:

- ./app:/usr/src/app

working_dir: /usr/src/app

command: ["node", "server.js"]

redis:

image: redis:alpine

container_name: redis

ports:

- "6379:6379"

คำอธิบาย

- เปิดพอร์ต 3000 สำหรับ WebSocket
- ใช้ Redis เป็นฐานข้อมูลชั่วคราว

อ้างอิง

1. <https://socket.io/>
2. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
3. <https://nodejs.org/>
4. <https://hub.docker.com/>
5. <https://expressjs.com/>